

第 16 章 RPM 包、文件归属与软件事务

第 16 章 RPM 包、文件归属与软件事务

先抓住一条主线：软件包问题不是“装没装”一个判断，而是包文件、RPM 数据库、磁盘文件、配置迁移和最终功能五个层次连续推进。

- 01识别包身份用 NEVRA 区分名称、版本、发行与架构
- 02选择查询对象区分已安装记录、本地 RPM 与磁盘路径
- 03建立文件归属从包查文件，从路径反查已安装包
- 04解释验证差异逐字符读取 rpm -V，不把差异直接判恶
- 05审阅软件事务由 DNF 解析依赖并确认完整事务摘要
- 06推进到真实终态检查配置迁移、unit、应用语法与功能

专题地图

知识专题	从包名到 NEVRA：先确认正在谈论哪个包
知识专题	RPM 数据库、磁盘文件与应用状态是三个层次
操作专题	查询已安装包并建立文件归属证据
操作专题	在安装前检查本地 RPM 包文件
操作专题	使用 rpm -V 解释文件差异，而不是机械修复
知识专题	.rpmnew 与 .rpmsave：升级时如何保护本地配置
操作专题	安装、升级、重装与删除：先读事务计划
操作专题	事务完成后，从包状态推进到真实功能
诊断专题	包事务和文件状态异常的证据链
经典任务	解释文件归属与验证差异；预检本地 RPM 并安装验收

阅读时持续回答

1. 当前查询对象是包名、本地 RPM 文件还是磁盘路径？
2. 这条证据来自 RPM 数据库，还是来自当前磁盘状态？
3. NEVRA 的哪个字段决定当前对象的精确身份？
4. rpm -V 的字符表示什么差异，文件类别是什么？
5. 该差异是合法配置、意外漂移，还是尚不能判断？

6. 事务摘要是否包含超出题目目标的安装、升级或删除？
7. `.rpmnew/.rpmsave` 是否需要比较、合并和语法验证？
8. 包事务成功后，应用或服务功能是否真正成立？

学习提示：先确定查询对象与证据层，再选择命令；看到差异时先解释字段和文件类别，最后才决定是否修改。完成事务后继续验证配置、unit 与真实功能。

第 16 章 · 正文

在 Linux 系统中，“软件已经装上”不是一个可以只靠一条命令证明的结论。一个 `.rpm` 文件可能只是保存在磁盘上，尚未写入系统；RPM 数据库可能记录某个包已经安装，但它声明拥有的文件可能被删除、改写或改变权限；DNF 事务可能已经成功结束，但新配置尚未合并、相关 unit 没有进入题目要求的状态，应用也可能仍无法提供真实功能。

本章围绕“包身份 → 查询对象 → 文件归属 → 差异验证 → 软件事务 → 功能终态”推进。最常见的误判有三类：把本地 RPM 文件存在当成已经安装；把 `rpm -V` 的差异直接判定为恶意或故障；把 DNF 的 `Complete!` 当成配置、服务和业务功能都已经正确。仓库、模块流与包组留给第 17 章《DNF 仓库、模块流与包组》；服务启动和持久状态留给第 12 章《systemd Unit、服务与依赖关系》。本章只建立完成当前软件事务所必需的接口，并明确不把 `--nodeps`、`--force` 或跳过签名校验当作正常解决方案。

概念

NEVRA 是 RPM 包的精确身份，由 Name、Epoch、Version、Release 和 Architecture 组成。包名只描述逻辑对象，本地文件名还可能被任意重命名，命令名和服务名也不等同于包名；因此遇到版本、架构或并行安装问题时，应让 RPM 从包头或数据库读取真实字段，而不是靠文件名或字符串外观猜测。

概念

RPM 数据库保存系统已经登记的包头、文件清单和文件属性基线，它回答“系统记录安装了什么”。数据库记录不等于磁盘实时镜像：文件可能被删除或修改，挂载也可能遮蔽路径；更不能由此直接推出应用配置、进程和业务功能正确。

概念

包文件归属 是已安装包与路径之间的声明关系。``rpm -ql`` 从包走向文件，``rpm -qf`` 从当前路径反查拥有者；归属只能说明哪个已安装包声明提供该路径，不说明谁最后修改了文件，也不能查询尚未安装的仓库候选。

概念

配置文件标记 是包作者赋予文件的升级保护语义。被标记为配置文件的路径可能包含合法本地修改；升级时 RPM 会根据包内标记、当前内容和新旧版本关系保留正式文件或旁置另一份，因此 ``.rpmnew`` 与 ``.rpmsave`` 是需要比较和迁移的证据，不是可以机械删除的垃圾文件。

概念

验证差异 是当前磁盘文件与 RPM 数据库基线之间的可检测变化。``rpm -V`` 可以指出大小、模式、摘要、所有者、组、时间戳或 capability 等差异，但它不判断变化是否合理；配置变更可能是预期状态，关键文件无差异也不等于服务或业务功能健康。

概念

软件事务 是一组需要整体审阅和提交的包变更。安装、升级、重装或删除可能牵动多个依赖、运行脚本并改变配置和 unit；DNF 负责选择候选和解析依赖，RPM 执行底层包操作并维护数据库。事务成功只是包层终态，仍需继续验证文件、配置、服务和功能。

操作语义以下命令分别回答“已安装什么、包里有什么、路径归谁、文件哪里不同、包是否可信、事务会改变什么”。先理解作用对象，再记参数。

`rpm -q` 查询已安装包

SYNOPSIS

```
rpm -q [QUERY-OPTIONS] PACKAGE...
```

以本机 RPM 数据库为查询对象，确认安装记录、包头信息和包声明的文件。

重要参数 / 形式

`-q`

进入查询模式；后续选项决定查询哪个对象和显示哪些信息。

`-i`

显示包头的详细信息，例如版本、架构、许可证、安装日期和描述。

`-l`

列出包记录的全部路径。

`-c`

只列出被包标记为配置文件的路径。

`-d`

只列出被包标记为文档的路径。

`rpm -qp` 查询本地 RPM 文件

SYNOPSIS

```
rpm -qp [QUERY-OPTIONS] ./PACKAGE_FILE.rpm
```

`-p` 把查询对象从已安装数据库切换为一个尚未安装的包文件；它适合安装前读取真实包头和内容。

重要参数 / 形式

`-qpi`

查看本地 RPM 的详细元数据，不执行安装。

`-qpl`

预览包将声明安装的路径。

`-qpc`

预览被标记为配置文件的路径。

`--requires`

读取包头中的依赖要求；查询本身不解析或安装依赖。

`--scripts`

查看包脚本内容；查询不会运行这些脚本。

`rpm -qf` 从路径反查已安装包

SYNOPSIS

```
rpm -qf /ABSOLUTE/PATH
```

查询哪个已安装包声明拥有当前路径。它只查本机 RPM 数据库，不能发现尚未安装的仓库候选。

重要参数 / 形式

`-f PATH`

把文件路径作为查询键；优先使用已确认的真实绝对路径。

`command -v NAME`

先确定外部命令路径；若结果是 alias、函数或 builtin，应回到第 03 章继续解析。

```
rpm -qf "$(command -v NAME)"
```

仅在 `command -v` 返回真实外部文件路径时组合使用。

`rpm -V` 验证已安装文件差异

SYNOPSIS

```
rpm -V PACKAGE
rpm -Vf /ABSOLUTE/PATH
```

比较当前文件与 RPM 数据库中的属性基线。默认无输出表示本次可检查属性未发现差异；有输出时必须逐字符解释，并结合文件类别判断下一条证据。

重要参数 / 形式

`-V PACKAGE`

验证指定已安装包的文件。

`-Vf PATH`

先按路径找到拥有包，再验证该文件范围。

`S M S D L U G T P`

依次表示大小、模式、文件摘要、设备、符号链接、所有者、组、修改时间和 capability 等差异位置。

`missing`

包清单记录路径，但当前视图中路径缺失；继续调查版本、挂载、误删或替换。

`rpmkeys --checksig` / `rpm -K` 检查包文件

SYNOPSIS

```
rpmkeys --checksig ./PACKAGE_FILE.rpm
rpm -K ./PACKAGE_FILE.rpm
```

检查包文件的摘要和签名状态。缺少公钥、包未签名和签名或摘要失败是不同结果；该检查也不能替代已安装文件的 `rpm -V`。

重要参数 / 形式

`--checksig FILE`

读取包文件中的摘要和签名并尝试验证。

`-K FILE`

RPM 命令提供的兼容入口；正文以 `rpmkeys` 作为语义更明确的首选写法。

真实输出

按环境记录实际状态，不把“缺少公钥”自动写成“签名失败”。

`dnf` 形成依赖事务

SYNOPSIS

```
dnf install ./PACKAGE_FILE.rpm
dnf upgrade PACKAGE
dnf reinstall PACKAGE
dnf remove PACKAGE
```

DNF 选择候选、解析依赖并显示完整事务计划；确认前应检查将安装、升级和删除哪些包。RPM 的 `-U`、`-F` 和 `-e` 用于理解底层语义，不作为绕开依赖处理的默认路径。

重要参数 / 形式

```
install ./FILE.rpm
```

以本地包文件为目标，同时使用已配置仓库满足依赖。

```
upgrade PACKAGE
```

推进到当前可用的更新候选；应记录最终 NEVRA。

```
reinstall PACKAGE
```

重装相同包内容，适合确认文件意外缺失或损坏后的受控恢复，不用于覆盖未知配置。

```
remove PACKAGE
```

删除目标并展示相关依赖变更；摘要范围异常时应取消。

```
rpm -U / -F / -e
```

分别表示低层升级或安装、仅 freshen 已安装包、删除；不得用 `--nodeps` 解决依赖。

[知识专题] 从包名到 NEVRA：先确认正在谈论哪个包

软件包调查最容易出现的错误，是把“名字看起来一样”当成“对象完全相同”。同一个 Name 可能有不同版本、发行号和架构；某些包允许多个版本并存；一个本地 `.rpm` 文件也可能比系统中已安装的版本更新、相同或更旧。只有把身份维度拆开，后续的安装、升级和删除才不会误伤其他对象。

① [知识点] NEVRA 的五个字段分别承担什么职责

字段	含义	判断边界
Name	包的逻辑名称	不等同于命令名或服务名
Epoch	版本比较的额外优先级	常为未设置；一旦设置会参与比较
Version	上游软件版本	不能只按字符串直觉比较
Release	打包发行号	同一上游版本可有多多个发行修订
Architecture	目标架构	<code>noarch</code> 表示内容不绑定特定 CPU 架构

常见文件名看起来像：

```
name-version-release.arch.rpm
```

但不要靠从右向左切连字符手工解析所有包名。Name、Version 和 Release 可能包含复杂字符；可靠做法是让 RPM 读取包头。

② [知识点] 包名、NEVRA、RPM 文件和路径是四种不同查询键

包名	→ 查询一个逻辑包对象
NEVRA	→ 精确到版本、发行和架构
FILE.rpm	→ 查询一个尚未安装的包文件
/PATH	→ 反查哪个已安装包声明拥有该路径

例如，`rpm -q <PACKAGE>` 查询 RPM 数据库；`rpm -qp <FILE.rpm>` 查询包文件；`rpm -qf <PATH>` 查询路径归属。把三者混用，常见结果是“package ... is not installed”或把文件名当成包名导致错误判断。

③ [知识点] Epoch 经常不可见，但不能从版本模型中删除

普通的 `rpm -q` 输出常把注意力集中在 Name、Version、Release 和 Architecture 上。Epoch 未设置时，很多显示形式会省略它；这不代表 Epoch 不参与 RPM 的版本比较模型。需要精确审计时，应使用查询格式分别读取字段，而不是从默认文本输出猜测。

```
rpm -q --qf 'Name: %{NAME}\nEpoch: %{EPOCH}\nVersion: %{VERSION}\nRelease: %\n{RELEASE}\nArch: %{ARCH}\n' <PACKAGE>
```

如果 Epoch 显示为未设置值，应记录真实结果，不自行改写成某个虚构数字。

④ [知识点] “已安装” 通常不是永久的一对一关系

大多数普通包在某一时刻只保留一个活动版本，但不能把这个经验扩大为规则。内核包是典型的可并行安装对象；多架构环境也可能存在同名的不同架构实例。执行删除、精确验证或脚本化查询前，应先查看所有匹配结果。

```
rpm -q <PACKAGE>
rpm -q --qf '%{NAME}\t%{EPOCH}\t%{VERSION}\t%{RELEASE}\t%{ARCH}\n' <PACKAGE>
```

第一条快速查看所有匹配记录；第二条逐字段保留每个匹配实例的真实身份。脚本不要通过拆分默认输出或模糊 `grep` 来猜测 NEVRA。

⑤ [知识点] RPM 文件名不是可信来源声明

文件名可以被任意重命名。真正的 Name、EVR、Architecture、Packager、Summary 和签名信息来自包头。收到第三方 RPM 时，先查询包头和签名，再决定是否允许进入事务。

[Cheatsheet] `-q` 查已安装数据库，`-qp` 查本地包文件，`-qf` 从路径反查已安装包；精确身份看 NEVRA，不看文件修改时间，也不只看文件名。

[知识专题] RPM 数据库、磁盘文件与应用状态是三个层次

RPM 数据库是包管理的事实入口，但它不是整个系统的实时镜像。数据库记录某包应该包含哪些文件、这些文件安装时具有哪些属性；磁盘上可能发生人工修改、配置管理变更、误删除或权限漂移；应用还可能因为配置语义、依赖服务或外部资源而失败。调查时必须明确自己正在证明哪一层。

① [知识点] RPM 数据库保存的是已安装记录和基线

数据库通常包含：

- 已安装包的包头和 NEVRA；

- 包声明安装的路径；
- 配置文件、文档、许可证等文件分类；
- 安装时用于验证的大小、模式、摘要、所有者、组、时间戳和 capability 等属性；
- 依赖、Provides、脚本等元数据。

本章通过 `rpm` 接口访问数据库，不依赖某个具体后端或内部数据库文件。RHEL 9 的实现细节可能随小版本变化，管理员脚本不应直接解析内部数据库文件。

② [知识点] 文件存在、文件归属和文件一致性是不同问题

```
ls /path 成功
只证明当前路径存在

rpm -qf /path 成功
证明某个已安装包声明拥有该路径

rpm -V <PACKAGE> 无输出
证明本次可检查属性未发现差异
```

它们都不能单独证明应用功能正确。配置文件内容可能与业务目标不符但仍与厂商包完全一致；服务也可能没有启动。

③ [知识点] `rpm -ql`、`-qc` 与 `-qd` 是全集和分类视图

- `rpm -ql <PACKAGE>`：列出包记录的全部路径；
- `rpm -qc <PACKAGE>`：只列出被标记为配置文件的路径；
- `rpm -qd <PACKAGE>`：只列出被标记为文档的路径。

不要用 `rpm -ql | grep /etc` 代替 `rpm -qc`。并非所有配置都一定位于 `/etc`，也不是 `/etc` 下的每个文件都被打包为 RPM 的配置文件。

④ [知识点] RPM 归属不等于“最后修改者”

`rpm -qf` 回答哪个已安装包声明拥有该路径，不回答谁最后修改了它。人工编辑、脚本、应用自身和配置管理系统都可能在安装后改变文件。要理解变化来源，需要结合时间、日志、配置管理记录和 `rpm -V` 差异。

⑤ [知识点] 路径不存在时仍可能在包清单中

一个受包管理的文件被删除后，`rpm -ql <PACKAGE>` 仍会列出它，而 `ls` 会失败，`rpm -V` 通常会报告 `missing`。这正说明数据库基线和磁盘现实是两个对象。

⑥ [知识点] 数据库一致性问题与普通文件漂移要分开

如果大量查询异常、RPM 数据库操作报错或事务中断，应先保留错误信息并调查数据库和事务状态。不要把所有问题都归因于某个文件被修改，也不要没有备份和官方恢复步骤时直接重建数据库。

[Cheatsheet] RPM 数据库回答“系统记录了什么”；磁盘检查回答“文件现在怎样”；应用验证回答“功能是否成立”。任何一层成功都不能替代下一层。

[操作专题] 查询已安装包并建立文件归属证据

查询的目标不是输出越多越好，而是让每条命令回答一个明确问题：包是否安装、精确身份是什么、安装了哪些路径、哪些是配置、某个路径归谁。调查应从包身份和查询方向开始，再进入文件和验证。

① [操作] 使用 `rpm -q` 和 `rpm -qi` 确认安装记录

作用对象：本机 RPM 数据库中的已安装包。

基本语义：

```
rpm -q <PACKAGE>
rpm -qi <PACKAGE>
```

`rpm -q` 适合快速确认安装状态和默认身份输出；`rpm -qi` 展示更完整的包头信息，如 Summary、Description、License、Install Date 和 Source RPM。

验证与边界：查询成功只证明数据库中有匹配记录。若需要精确区分架构、Epoch 或多个版本，应使用查询格式并保留所有匹配行。

② [操作] 使用查询格式输出稳定字段

```
rpm -q --qf '%{NAME}\t%{EPOCH}\t%{VERSION}\t%{RELEASE}\t%{ARCH}\n' <PACKAGE>
```

查询格式适合审计和脚本化，但字段内容仍需按实际输出处理。不要用空格切割默认的 `rpm -q` 文本来推断复杂包名。

可用标签可通过以下帮助入口查看：

```
rpm --querytags | less
man rpm
```

③ [操作] 从包查询全部路径、配置和文档

```
rpm -ql <PACKAGE>
rpm -qc <PACKAGE>
rpm -qd <PACKAGE>
```

调查安装后的配置入口时，先用 `-qc` 得到包作者声明的配置文件，再检查文件是否存在、是否产生旁置版本、内容是否满足业务目标。

④ [操作] 从路径反查已安装包

```
rpm -qf /absolute/path
```

先把命令解析为真实路径，再反查：

```
command -v <COMMAND>  
rpm -qf "$(command -v <COMMAND>)"
```

如果 `command -v` 返回 Shell builtin、函数或 alias，不能直接把它当成外部文件路径；命令发现的完整方法归第 03 章《本地帮助、命令发现与软件能力查询》。

⑤ [操作] 查询包脚本与依赖只做调查，不等于执行

```
rpm -q --scripts <PACKAGE>  
rpm -q --requires <PACKAGE>  
rpm -q --provides <PACKAGE>
```

这些命令读取已安装包的元数据。查看 scriptlet 有助于理解安装、升级和删除可能改变哪些系统对象，但查询本身不会运行脚本。

⑥ [操作] 处理“不属于任何包”的结果

`rpm -qf` 找不到所有者时，可能意味着：

- 文件由管理员或应用自行创建；
- 文件来自源码安装、手工解压或容器映射；
- 路径是运行时生成内容；
- 路径拼写、符号链接或挂载视图与预期不同；
- 提供该路径的包尚未安装。

若目标是查找可安装候选包，应转到第 17 章使用 DNF 的仓库查询接口，而不是强行让 `rpm -qf` 查询未安装内容。

[Cheatsheet] 包到文件： `-ql/-qc/-qd`；文件到包： `-qf`；脚本化身份： `--qf`；元数据查询不运行脚本。

[操作专题] 在安装前检查本地 RPM 包文件

本地 RPM 是一个尚未进入系统事务的候选对象。最安全的切入路径是：先确认文件和包头，再检查架构、内容、依赖、脚本和签名，最后让 DNF 形成事务计划。不要因为扩展名是 `.rpm` 就直接以 root 身份执行安装。

① [操作] 使用 `-p` 将查询对象切换到包文件

```
rpm -qp ./package.rpm
rpm -qpi ./package.rpm
rpm -qpl ./package.rpm
rpm -qpc ./package.rpm
rpm -qpd ./package.rpm
```

这里的 `-p` 是 package file 选择器。它告诉 RPM 读取指定文件的包头，而不是从已安装数据库中按包名查找。

② [操作] 明确检查 NEVRA 和目标架构

```
rpm -qp --qf '%{NAME}\t%{EPOCH}\t%{VERSION}\t%{RELEASE}\t%{ARCH}\n' ./package.rpm
uname -m
```

`noarch` 包通常不绑定特定 CPU 架构；其他架构必须与系统和运行环境相容。仅仅文件可读，不代表架构可安装。

③ [操作] 检查依赖、Provides 和脚本

```
rpm -qp --requires ./package.rpm
rpm -qp --provides ./package.rpm
rpm -qp --scripts ./package.rpm
```

包脚本可能在高权限下修改用户、目录、缓存、systemd preset 或配置。第三方 RPM 的脚本属于变更评审内容，不应跳过。

④ [操作] 使用 `rpmkeys --checksig` 检查摘要和签名

```
rpmkeys --checksig ./package.rpm
# 兼容入口
rpm -K ./package.rpm
```

该检查面向包文件中包含的摘要和数字签名，帮助判断文件完整性和签名来源。常见情况包括：

- 摘要和签名可成功验证；
- 包有签名，但本机缺少对应公钥；
- 包未签名；
- 摘要或签名验证失败。

必须记录真实输出。缺少公钥不等于包内容一定被篡改，未签名也不等于可信；两者都意味着无法用当前信任链完成来源验证。

⑤ [知识点] 签名检查与 `rpm -V` 不是同一种验证

```
rpmkeys --checksig FILE.rpm
```

比较包文件内部的摘要和签名

目标：包文件的完整性与来源

```
rpm -V PACKAGE
```

比较已安装文件与 RPM 数据库基线

目标：安装后文件属性是否漂移

一个签名有效的包安装后仍可能被人工修改；已安装文件完全匹配基线，也不能证明这个包来自组织允许的来源。

⑥ [操作] 必要时只提取内容，不执行事务

RH124 课程展示了 `rpm2cpio` 与 `cpio` 用于检查或提取包内容。它适合离线审阅单个文件，但不是本章安装事务的主路径。

```
mkdir inspect && cd inspect
```

```
rpm2cpio ../package.rpm | cpio -t
```

```
rpm2cpio ../package.rpm | cpio -idmv './path/in/package'
```

提取出来的文件不进入 RPM 数据库，也不会执行 scriptlet；因此不能把这种提取视为“安装完成”。

⑦ [操作] 形成安装前决策记录

文件路径和权限已确认

→ 包头身份已确认

→ 架构已确认

→ 文件、配置和脚本已审阅

→ 签名策略已满足

→ 当前安装版本已确认

→ DNF 事务摘要可接受

[Cheatsheet] `-p` 查询包文件；`--scripts` 只查看脚本；`rpmkeys --checksig` 检查包文件；提取内容不等于安装。

[操作专题] 使用 `rpm -V` 解释文件差异，而不是机械修复

RPM 验证将当前文件系统状态与数据库记录的基线进行比较。它非常适合回答“哪些受包管理的属性发生了变化”，但输出只是一组差异标记。管理员仍需判断变化是否预期、是否来自配置管理、是否影响功能，以及最小修复是什么。

① [操作] 选择验证范围

```
rpm -V <PACKAGE>          # 验证一个已安装包
rpm -Vf /absolute/path    # 通过路径选择所属包并验证
rpm -Va                    # 验证全部已安装包，范围较重
```

优先选择最小范围。全系统验证可能产生大量预期配置差异，也会增加分析成本。

② [知识点] 无输出的准确含义

`rpm -V <PACKAGE>` 在本次可比较属性均匹配时通常不输出文件行。它只能证明：

对所选包、本次未禁用且可读取的验证属性，没有发现与 RPM 数据库基线不同的项目。

它不能证明：

- 配置符合业务要求；
- 服务正在运行或开机自启；
- 端口可达；
- 数据内容正确；
- 包来源符合组织信任策略。

③ [输出判断] 读取九个属性位置

典型差异行的前九个位置为：

S M 5 D L U G T P

位置	含义	下一条证据示例
S	文件大小变化	stat、diff、应用生成逻辑
M	模式或文件类型变化	stat -c '%A %a %F'
5	文件摘要变化	diff、版本控制、配置管理记录
D	设备号变化	stat，确认特殊文件语义
L	符号链接目标变化	readlink -f、ls -l
U	所有者变化	stat -c '%U %u'
G	所属组变化	stat -c '%G %g'
T	修改时间变化	stat，不能单独判定内容变化
P	capability 变化	getcap、安全基线

字符 `.` 表示该位置未发现差异；`?` 表示该项无法完成检查。`5` 是历史沿用的显示字符，现代解释宜称为文件摘要差异，不应机械理解为所有版本都只使用 MD5。

④ [输出判断] 读取文件属性标记和路径

九位属性之后通常还有文件类别标记和路径，例如：

- `c`：配置文件；
- `d`：文档文件；
- `g`：ghost 文件；
- `l`：许可证文件；
- `r`：README 文件。

实际可见标记受 RPM 版本和包定义影响。判断时先读差异位置，再读文件类别，最后看路径；不要只看到 `c` 就断言配置错误。

⑤ [输出判断] `missing` 与属性漂移是两种证据

`missing` 表示数据库记录的路径当前不存在。调查链应是：

```
确认包仍安装
→ 确认路径在 rpm -ql 中
→ 检查父目录、挂载和符号链接
→ 查明删除或替换来源
→ 评估是否可用 dnf reinstall 恢复
→ 重新验证并做功能检查
```

不要在不知道本地定制和业务状态时直接重装整个包。

⑥ [诊断] 配置文件差异先判断是否预期

配置文件出现 `5`、`S`、`T` 等差异很常见。下一步优先：

```
rpm -qc <PACKAGE>
stat <CONFIG>
diff -u <KNOWN_BASELINE> <CONFIG>
```

如果没有独立基线，可与 `.rpmnew`、备份、版本控制或同版本包中的文件进行比较。`rpm -V` 只告诉你“不同”，不告诉你哪一方符合业务目标。

⑦ [安全边界] 不把验证差异自动等同于入侵

差异可能来自：

- 合法配置变更；
- 应用首次运行生成内容；

- 管理工具修正权限；
- 升级遗留；
- 人工误操作；
- 未授权修改。

判断必须结合变更记录、日志、所有者、时间线和应用行为。验证输出是调查入口，不是结论。

[Cheatsheet] `rpm -V` 只报告差异；先读九位属性，再读文件类别和路径；无输出不是功能验收，配置差异也不是自动重装理由。

[知识专题] `.rpmnew` 与 `.rpmsave`：升级时如何保护本地配置

升级软件时，包中的配置文件可能与当前系统上的本地修改冲突。RPM 不总是简单覆盖原文件，而会依据包作者的配置标记、当前文件是否被修改以及新旧内容关系，保留当前文件或旧版本。旁置文件不是垃圾，它们是配置迁移证据。

① [知识点] 配置文件保护依赖打包标记和实际状态

包作者可使用配置文件标记控制升级行为。管理员无法只凭扩展名预测所有包的结果。讲义中的 `.rpmnew` 和 `.rpmsave` 描述常见模式，不把它们写成对所有 RPM 都成立的固定公式。

② [知识点] `.rpmnew` 的常见语义

对于通常被标记为“不直接替换”的配置文件，当当前文件已被本地修改且新包提供不同内容时，当前正式路径往往保留，新厂商版本被写为：

```
/path/to/config.rpmnew
```

这表示新版本候选尚未合并，不表示升级事务一定失败。

③ [知识点] `.rpmsave` 的常见语义

在另一类配置替换策略下，新包版本可能取得正式路径，而原有本地文件被保存为：

```
/path/to/config.rpmsave
```

这时需要从旧文件中迁移仍然有效的本地设置，不能只检查正式路径是否存在。

④ [操作] 系统化定位旁置文件

针对事务涉及的目录优先查找，避免无目的扫描整个系统：

```
find /etc -type f \( -name '*.rpmnew' -o -name '*.rpmsave' \) -print
```


还应结合：

```
rpm -qc <PACKAGE>
dnf history info <ID>
```

从事务和包清单确认旁置文件与本次升级的关系。

⑤ [操作] 比较、合并、语法检查，再让应用加载

```
确认当前正式文件和旁置文件
→ diff -u
→ 识别已废弃和新增参数
→ 选择性合并
→ 应用自身的配置语法检查
→ 备份后原子替换
→ 按第 12 章入口 reload 或 restart
→ 功能验证
```

不要直接 `cp config.rpmnew config` 覆盖现有业务配置，也不要未确认新版本语义时把 `.rpmnew` 全量复制回来。

⑥ [边界] 删除旁置文件必须是处理结果，不是清理动作

只有在以下条件都成立后，才考虑删除：

- 已完成差异审阅；
- 需要保留的设置已迁移；
- 新配置语法检查通过；
- 应用和服务验证通过；
- 已有可回退备份或版本记录。

[Cheatsheet] `.rpmnew` 常见为“保留当前，旁置新版本”；`.rpmnew` 常见为“正式路径使用新版本，保存旧文件”。无论哪种都必须 diff、合并、验证。

[操作专题] 安装、升级、重装与删除：先读事务计划

包管理操作会改变 RPM 数据库、文件系统和依赖关系。RHEL 9 中，普通管理任务优先使用 DNF；低层 RPM 选项用于理解语义、查询和特殊调查，而不是绕过依赖保护。任何删除或大范围升级都应先审阅事务摘要。

① [知识点] RPM 与 DNF 的职责边界

DNF

- └─ 解析包规格和仓库候选
- └─ 处理依赖与冲突
- └─ 形成事务计划和摘要
- └─ 下载包并记录 history
- └─ 调用 RPM 完成底层事务

RPM

- └─ 读取包头与本地数据库
- └─ 安装、升级、删除单个包事务
- └─ 执行 scriptlet 和触发器
- └─ 更新 RPM 数据库
- └─ 查询与验证包及文件

因此，本地 RPM 也通常用 DNF 安装：DNF 可以利用已配置仓库补齐依赖。

② [操作] 通过推荐路径安装本地 RPM

```
sudo dnf install ./package.rpm
```

`./` 或绝对路径帮助 DNF 明确这是本地文件，而不是仓库中的包名。执行前应阅读：

- 安装、升级、删除哪些包；
- 依赖和弱依赖；
- 下载和安装大小；
- 是否出现冲突、替换或架构异常。

考试中不应为了省时间盲目使用 `-y` 跳过摘要。

③ [操作] 区分安装、升级和重装

```
sudo dnf install <PACKAGE>
sudo dnf upgrade <PACKAGE>
sudo dnf reinstall <PACKAGE>
```

- `install`：使指定内容进入已安装状态，必要时安装依赖；
- `upgrade`：选择可用的较新版本并处理依赖；
- `reinstall`：重新安装当前可获得的相同包，常用于恢复缺失或损坏的受管理文件；
- 任何操作后都要验证真实 NEVRA 和文件状态。

`reinstall` 不是清除所有本地配置的工具，也不能自动修复应用数据和外部依赖。

④ [知识点] 低层 RPM 安装选项表达不同事务语义

```
rpm -i FILE.rpm      # 安装，不承担一般升级选择
rpm -U FILE.rpm      # 安装或升级
rpm -F FILE.rpm      # 仅当同名包已安装时更新
rpm -e PACKAGE       # 删除已安装包
rpm --test ...       # 进行事务测试，不应用变更
```

课程可能展示 `rpm -ivh` 以说明低层安装。真实 RHEL 9 管理中，依赖可由仓库解决时仍优先 DNF。

⑤ [操作] 删除前检查完整事务摘要

```
sudo dnf remove <PACKAGE>
```

DNF 可能计划删除依赖该包的其他内容，或清理不再需要的依赖。看到删除数量超出预期时应取消，回到：

```
rpm -q --whatrequires <PACKAGE>
dnf history
```

仓库级依赖分析与候选查询在第 17 章深入。

⑥ [操作] 使用 history 建立事务审计线索

```
dnf history
dnf history info <ID>
```

重点查看：

- 命令行和时间；
- 事务成功、失败或中止状态；
- 安装、升级、降级、删除的包；
- 事务前后数据库是否出现外部变化提示。

`dnf history undo <ID>` 依赖旧版本包仍可获得、仓库状态和后续依赖关系，不能当作无条件回滚按钮。

⑦ [安全边界] 这些选项不是默认修复方法

```
rpm --nodeps
rpm --force
rpm --replacefiles
rpm --oldpackage
dnf --allowerase
dnf --nogpgcheck
```

它们会放宽依赖、文件冲突、版本或签名保护。遇到失败时应先读取准确错误，确认是依赖缺失、冲突、签名、架构还是仓库问题，再选择最小修复。仓库配置和 GPG 策略归第 17 章。

⑧ [边界] 包管理事务与服务控制不是同一步

某些包脚本会创建用户、刷新缓存或应用 preset，但不能假设安装后服务必然启动、启用或提供功能。事务完成后进入 O05 的分层验收；完整 systemd 控制归第 12 章。

[Cheatsheet] 普通事务优先 DNF；本地包用路径安装；删除先读摘要；history 是证据，不是保证；不以 `--nodeps`、`--force` 或关闭 GPG 校验换取“成功”。

[操作专题] 事务完成后，从包状态推进到真实功能

DNF 显示事务完成，只能证明包管理层没有报告失败。考试和工作中的终态通常还包含精确版本、关键文件、配置合并、服务状态和实际功能。验收应从最接近事务的证据开始，逐层向外推进。

① [验证] 第一层：确认 RPM 数据库中的最终身份

```
rpm -q <PACKAGE>
rpm -q --qf '%{NAME}\t%{EPOCH}\t%{VERSION}\t%{RELEASE}\t%{ARCH}\n' <PACKAGE>
```

确认实际安装的是目标包、目标版本和目标架构。不要只保留 DNF 最后一行文本。

② [验证] 第二层：确认关键路径和文件归属

```
rpm -ql <PACKAGE>
rpm -qc <PACKAGE>
rpm -qf <CRITICAL_PATH>
```

同时使用 `test -e`、`stat` 或 `readlink` 检查关键路径现实。RPM 清单存在而磁盘路径缺失时，应继续验证。

③ [验证] 第三层：检查文件漂移和配置旁置文件

```
rpm -V <PACKAGE>
find <RELEVANT_DIR> -type f \( -name '*.rpmnew' -o -name '*.rpmsave' \) -print
```

对已知会本地修改的配置，预期差异应有解释；对关键可执行文件或 unit 的意外差异应继续调查。

④ [验证] 第四层：运行应用自身的静态检查

按应用提供的接口检查：

```
<COMMAND> --version
<COMMAND> --help
<APPLICATION> --config-test
```

具体命令取决于包。不要编造不存在的 `--config-test`；应从 man page、`--help` 或包文档确认真实入口。

⑤ [验证] 第五层：检查 unit、进程和功能

若包提供 systemd unit：

```
rpm -ql <PACKAGE> | grep '/systemd/system/'
systemctl status <UNIT>
systemctl is-active <UNIT>
systemctl is-enabled <UNIT>
```

`active`、`enabled` 和真实业务功能是不同的状态。完整启停、依赖和持久化控制归第 12 章《systemd Unit、服务与依赖关系》。

⑥ [验证] 第六层：保留事务和变更证据

```
dnf history info <ID>
```

记录：目标、实际包、时间、事务结果、配置处理、验证命令和仍未确认的风险。真实运维中，这些信息可成为后续 Ansible 自动化的前置基线。

⑦ [工作迁移] 把手工变更转化为幂等状态

后续自动化不应简单复制一条 `dnf install` 命令，而应表达：

- 目标包和允许版本
 - 依赖与仓库前提
 - 配置文件来源和所有权
 - 需要 `reload/restart` 的条件
 - 包层、文件层和功能层验证

Ansible 模块和仓库自动化属于 RHCE 章节，本章只建立可自动化的状态模型。

[Cheatsheet] 事务成功后依次验证：NEVRA → 关键路径 → `rpm -V` 和旁置配置 → 应用静态检查 → unit/进程 → 真实功能 → 事务记录。

[诊断专题] 包事务和文件状态异常的证据链

软件管理故障经常跨越包文件、数据库、依赖、配置和服务。稳定的诊断方法不是反复重装，而是先识别症状属于哪一层，再选择最有区分度的证据。

① [诊断] 本地 RPM 无法安装

症状: `dnf install ./package.rpm` 失败

- 当前证据: 保留完整错误和事务摘要
- 假设: 文件损坏、签名、公钥、架构、依赖或冲突
- 下一条证据: `rpm -qpi`、`rpmkeys --checksig`、`rpm -qp --requires`、`uname -m`
- 最小修复: 补齐可信前提或选择正确包，不绕过保护
- 再验证: 重新审阅事务摘要和最终 NEVRA

仓库不可用、GPG key 配置或模块流问题转第 17 章。

② [诊断] 包显示已安装，但关键文件不存在

- ```
rpm -q <PACKAGE>
```
- `rpm -ql <PACKAGE> | grep <PATH>`
  - `ls -l / stat / findmnt`
  - `rpm -V <PACKAGE>`
  - 调查删除、挂载覆盖或路径替换来源
  - 确认后再考虑 `dnf reinstall`
  - 应用和功能再验证

先确认该路径确实由当前版本包声明；版本变化可能使旧路径合法消失。

### ③ [诊断] `rpm -V` 出现大量配置差异

先按文件类别和目录分组，区分：

- 预期本地配置；
- 版本升级带来的新语义；
- 配置管理系统维护；
- 意外权限或所有权漂移；
- 内容被覆盖或删除。

下一条证据优先是 `rpm -qc`、`stat`、`diff`、变更记录和应用语法检查，而不是 `dnf reinstall`。

#### ④ [诊断] 升级成功，但服务无法读取新配置

```
查找 .rpmnew/.rpmsave
→ 确认当前正式配置由哪个版本产生
→ diff 并识别废弃参数
→ 应用配置语法检查
→ 最小合并
→ reload/restart
→ 日志和功能验证
```

服务控制和日志入口分别引用第 12 章与第 13 章，不在本章重新展开。

#### ⑤ [诊断] 删除事务计划移除大量包

```
症状: remove 摘要远超目标范围
→ 立即取消
→ 记录哪些包因依赖被计划删除
→ rpm -q --whatrequires 与 dnf history 调查
→ 重新确认真正目标和替代依赖
→ 不使用 --nodeps 强行删除
```

摘要本身就是重要证据，取消事务不是失败，而是避免破坏性变更。

#### ⑥ [诊断] DNF 显示成功，但命令仍不可用

```
rpm -q 确认包
→ rpm -ql 查包是否真的提供该命令
→ command -v 和 PATH
→ rpm -qf 反查实际文件
→ stat / rpm -V
→ Shell 缓存、权限和解释器
→ 应用级验证
```

如果目标命令由另一个包提供，转第 17 章使用仓库能力查询。

#### ⑦ [诊断] 签名验证提示缺少公钥

缺少公钥表示当前信任库无法验证签名来源。应确认：

- 包来自哪个受控渠道；
- 正确公钥的可信分发位置；
- 指纹是否通过独立渠道核对；
- 组织策略是否允许导入。

不要从与包同一未知来源随手下载一个 key 后立即信任，也不要以 `--nogpgcheck` 跳过问题。

⑧ [诊断] history 中的 undo 无法完成

可能原因包括旧包不再可用、仓库内容变化、后续事务改变依赖或 RPM 数据库有外部变化。下一步应读取错误中具体缺失的包和版本，评估恢复来源和影响，不应反复执行 undo 或用 `--allowerase` 扩大变更。

[Cheatsheet] 先保存错误和摘要；按包文件、签名、架构、依赖、数据库、文件、配置、服务分层；选择最小修复，再回到同一层复查。

⑨ [诊断] 用最有区分度的证据切开故障层次

| 症状                         | 优先取得的下一条证据                                                                                           | 此时不要直接做什么                                             |
|----------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 本地 RPM 无法进入事务              | <code>rpm -qpi</code> 、 <code>rpmkeys --checksig</code> 、 <code>rpm -qp --requires</code> 与完整 DNF 摘要 | 不使用 <code>--nodeps</code> 或 <code>--nogpgcheck</code> |
| 包已安装但关键路径缺失                | <code>rpm -ql</code> 、父目录与挂载、 <code>rpm -Vf</code>                                                   | 不在未解释缺失原因时反复重装                                        |
| <code>rpm -V</code> 出现配置差异 | 文件类别、 <code>stat</code> 、 <code>diff</code> 、变更记录与应用语法检查                                             | 不把所有配置差异当成损坏                                          |
| 升级后应用不可用                   | <code>.rpmnew/.rpmsave</code> 、应用静态检查、unit 状态与日志                                                     | 不把 <code>Complete!</code> 当成功能终态                      |
| remove 或 undo 计划异常扩大       | 完整事务摘要、 <code>--whatrequires</code> 、history 与缺失版本                                                   | 不用放宽冲突保护强行提交                                          |

这张表只负责选择下一条证据；实际修复仍应回到对应对象层，完成最小变更并重新验证。

[经典任务] 任务一：识别文件所属包并解释验证差异

环境

你在一台 RHEL 9 主机上维护 `websvc` 应用。题目环境会提供两个真实路径：

```
<CONFIG_PATH>
<CRITICAL_PATH>
```

两个路径均应来自同一个已安装 RPM 包，但题目不会直接给出包名。配置文件曾经过合法本地修改；关键路径可能存在内容、权限、所有者或缺失问题。

当前状态

- 不允许假设 `rpm -V` 只输出一行；
- 不允许把所有配置差异当作故障；
- 不知道最近修改来自人工、脚本还是升级；
- 本任务不要求配置仓库，也不要求制作 RPM。



## 目标终态

1. 识别两个路径所属的已安装包；
2. 记录该包的真实 Name、Epoch、Version、Release 和 Architecture；
3. 列出该包的配置文件、文档和关键文件；
4. 对包和指定路径执行有针对性的验证；
5. 逐条解释验证差异字段、文件类别和下一条证据；
6. 对预期配置变化不做破坏性覆盖；
7. 对确认的意外漂移给出最小修复方案；
8. 完成包层、文件层、配置层和功能层验收。

## 限制条件

- 不使用 `rpm --force`、`--nodeps` 或无调查的重装；
- 不把 `rpm -V` 无输出扩大为服务健康；
- 不编造路径、包名或验证输出；
- 修改配置前必须保留备份或差异记录。

## 验收证据

| 层次   | 证据                                                                      |
|------|-------------------------------------------------------------------------|
| 归属   | <code>rpm -qf</code>                                                    |
| 精确身份 | <code>rpm -q --qf</code>                                                |
| 文件分类 | <code>rpm -ql/-qc/-qd</code>                                            |
| 差异   | <code>rpm -V</code> 、 <code>rpm -Vf</code>                              |
| 当前属性 | <code>stat</code> 、 <code>readlink</code> 、 <code>getcap</code> （按差异选择） |
| 内容   | <code>diff</code> 或受控基线                                                 |
| 应用   | 配置语法和功能验证                                                               |

## 作答记录模板

| 对象              | 已取得的基线                           | 差异解释与下一条证据                                                                                  | 再验证结果     |
|-----------------|----------------------------------|---------------------------------------------------------------------------------------------|-----------|
| <CONFIG_PATH>   | 归属、文件类别、当前属性                     | 记录预期配置或意外漂移，并选择 <code>diff</code> / 语法检查                                                    | 保留真实结果    |
| <CRITICAL_PATH> | 归属、包清单、当前存在性                     | 按 <code>rpm -Vf</code> 字段选择 <code>stat</code> / <code>readlink</code> / <code>getcap</code> | 保留真实结果    |
| <PACKAGE>       | NEVRA、 <code>rpm -V</code> 与配置清单 | 汇总是否需要最小修复，不扩大事务范围                                                                          | 包层与功能层均复查 |

这个模板用于把“看到差异”转成可审计的判断链；答案不能只贴命令而不解释证据。

## [参考解答] 任务一：从归属调查到最小修复

下面使用占位符表示题目提供的真实路径。执行时必须替换为实际对象，并保留真实输出。

### ① 调查路径归属，不先猜包名

```
rpm -qf <CONFIG_PATH>
rpm -qf <CRITICAL_PATH>
```

如果两条结果不同，应停止套用“同一包”的假设，分别调查；如果某条路径不存在，先用父目录、题目说明和候选包清单确认路径，再通过包清单判断是否应存在。

将共同包名保存为逻辑占位符：

```
pkg=$(rpm -qf <CONFIG_PATH>)
printf '%s\n' "$pkg"
```

注意：`rpm -qf` 默认可能输出完整包标识。后续命令可直接使用该结果，也可另行查询 Name 字段。

### ② 记录精确包身份

```
rpm -q --qf 'Name: %{NAME}\nEpoch: %{EPOCH}\nVersion: %{VERSION}\nRelease: %
{RELEASE}\nArch: %{ARCH}\n' "$pkg"
```

验收时保留真实字段。Epoch 未设置时不虚构值。

### ③ 建立包文件视图

```
rpm -ql "$pkg"
rpm -qc "$pkg"
rpm -qd "$pkg"
```

确认 `<CONFIG_PATH>` 是否被标记为配置文件，确认 `<CRITICAL_PATH>` 是否仍在当前版本文件清单中。若路径不在清单中，它可能来自旧版本、运行时生成或其他对象。

### ④ 先验证最小范围，再决定是否扩大

```
rpm -Vf <CONFIG_PATH>
rpm -Vf <CRITICAL_PATH>
rpm -V "$pkg"
```

记录每条输出：

- 九位属性差异
- 文件类别标记
- 路径
- 当前是否存在

若无输出，只记录“本次可检查属性未发现差异”，不写“服务正常”。

⑤ 根据差异选择下一条证据

```
stat <CONFIG_PATH>
stat <CRITICAL_PATH>
readlink -f <CRITICAL_PATH> # 仅在 L 差异或符号链接场景
getcap <CRITICAL_PATH> # 仅在 P 差异场景
```

内容摘要差异：

```
diff -u <KNOWN_BASELINE> <CONFIG_PATH>
```

如果没有独立基线，优先检查 `.rpmnew/.rpmsave`、版本控制、配置管理记录或从相同 NEVRA 包文件中提取对应文件，不把未知来源文件当作“正确版本”。

⑥ 区分预期配置与意外漂移

- 配置差异与已批准变更一致：记录为预期，不覆盖；
- 权限或所有者与应用要求不符：在确认业务和安全要求后做最小修正；
- 关键文件缺失或内容意外变化：确认没有本地定制后，可评估 `dnf reinstall "$pkg"`；
- 同一包大量文件异常：先调查事务、磁盘和安全事件，不机械逐文件修复。

⑦ 修复后回到同一证据层

```
rpm -V "$pkg"
```

预期配置仍可能继续显示差异，应在记录中解释。然后执行应用真实提供的配置语法检查和功能检查。

⑧ 验收矩阵

| 层次   | 合格条件        |
|------|-------------|
| 包身份  | NEVRA 与目标一致 |
| 文件归属 | 两个路径归属已明确   |

| 层次   | 合格条件              |
|------|-------------------|
| 文件差异 | 每条差异有解释或修复记录      |
| 配置   | 合法本地修改未被覆盖，语法检查通过 |
| 功能   | 应用的真实功能检查通过       |
| 安全   | 未使用放宽依赖和冲突保护的快捷选项 |

### 典型错误

- 用 `rpm -qa | grep` 的模糊结果直接进入删除或重装；
- 看到配置 5 差异就用厂商文件覆盖；
- 只运行 `rpm -q` 就宣布问题解决；
- 对 `missing` 不检查挂载、符号链接和版本变化；
- 修复后不再次运行同一验证命令。

### 提交前自检

- 两个路径的归属是否分别核实，而不是先假设同包；
- 是否记录真实 NEVRA，而不是从文件名或默认文本中猜字段；
- 每条 `rpm -v` 输出是否同时解释属性位置、文件类别和路径；
- 配置差异是否先与基线或变更记录比较；
- 最小修复后是否回到同一命令复查，并继续做应用功能验证；
- 报告中是否明确区分“未发现属性差异”和“服务功能健康”。

满足这些条件后，任务一才形成从调查、判断、修复到再验证的闭环。

## [经典任务] 任务二：检查本地 RPM 后以推荐路径安装并验收

### 环境

题目在以下目录提供一个本地 RPM：

```
/home/student/incoming/<PACKAGE_FILE>.rpm
```

系统已经有可用的 RHEL 9 仓库来满足依赖。本任务不得修改仓库配置；仓库和模块流归第 17 章。

### 当前状态

- 不确定同名包是否已经安装；
- 不确定本地 RPM 的版本、架构、脚本和签名状态；
- 包可能提供配置文件和 systemd unit；

- 不能假设安装后服务自动启动。

## 目标终态

1. 查询本地 RPM 的精确身份、摘要、文件、配置、依赖和脚本；
2. 检查包摘要和签名，并记录真实信任状态；
3. 查询系统当前是否安装同名包及其 NEVRA；
4. 使用 DNF 的本地文件路径执行适当安装或升级；
5. 完整阅读并确认事务摘要；
6. 验证最终包身份、关键文件和配置文件；
7. 检查 `.rpmnew/.rpmsave` 和 `rpm -V`；
8. 若包提供 unit，分别检查 unit 存在、当前状态、持久状态和真实功能。

## 限制条件

- 不使用 `rpm --nodeps`、`--force`、`dnf --nogpgcheck` 或 `--allowerase`；
- 不把缺少公钥自动改写成“签名失败”；
- 不用 `-y` 跳过第一次事务摘要审阅；
- 不编造服务名、端口或测试输出。

## [参考解答] 任务二：本地包预检、事务与分层验收

### ① 建立文件对象并确认可读

```
pkgfile=/home/student/incoming/<PACKAGE_FILE>.rpm
ls -l "$pkgfile"
file "$pkgfile"
```

`file` 只能作为外形检查，真正包身份仍由 RPM 包头提供。

### ② 查询本地包头和文件内容

```
rpm -qpi "$pkgfile"
rpm -qp --qf 'Name: %{NAME}\nEpoch: %{EPOCH}\nVersion: %{VERSION}\nRelease: %
{RELEASE}\nArch: %{ARCH}\n' "$pkgfile"
rpm -qpl "$pkgfile"
rpm -qpc "$pkgfile"
rpm -qp --requires "$pkgfile"
rpm -qp --scripts "$pkgfile"
```

记录 Name，供下一步查询已安装状态。不要从文件名猜 Name。

```
name=$(rpm -qp --qf '%{NAME}\n' "$pkgfile")
printf '%s\n' "$name"
```

### ③ 检查摘要、签名和架构

```
rpmkeys --checksig "$pkgfile"
uname -m
```

如果输出显示缺少公钥，应暂停并按组织信任流程取得正确公钥和核对指纹；本任务不允许用 `--nogpgcheck` 跳过。若包确实未签名，应按题目或组织策略决定是否允许，不能把它描述为已验证来源。

### ④ 查询当前已安装状态

```
rpm -q "$name"
rpm -q --qf '%{NAME}\t %{EPOCH}\t %{VERSION}\t %{RELEASE}\t %{ARCH}\n' "$name"
```

未安装、相同版本、较旧版本或并行架构会影响事务性质。不要只因为 `rpm -q` 非零就直接执行低层 `rpm -i`。

### ⑤ 通过 DNF 路径形成事务计划

```
sudo dnf install "$pkgfile"
```

在确认前检查：

- 目标包是安装还是升级；
- 需要新增、升级或删除哪些依赖；
- 是否出现意外架构或冲突；
- 总下载和安装大小；
- 是否有超出题目目标的删除。

摘要符合目标后再确认。若失败，保留错误并按 D01 分层调查，不添加破坏性选项。

### ⑥ 验证 RPM 数据库和关键路径

```
rpm -q "$name"
rpm -q --qf '%{NAME}\t %{EPOCH}\t %{VERSION}\t %{RELEASE}\t %{ARCH}\n' "$name"
rpm -ql "$name"
rpm -qc "$name"
```

从 `rpm -ql` 选择题目要求的关键路径：

```
rpm -qf <CRITICAL_PATH>
stat <CRITICAL_PATH>
```

## ⑦ 检查文件差异和配置迁移

```
rpm -V "$name"
find <RELEVANT_DIR> -type f \(-name '*.rpmnew' -o -name '*.rpmsave' \) -print
```

若存在旁置文件，执行 diff、选择性合并和应用语法检查，不机械覆盖。

## ⑧ 进入 unit 和功能层

先确认包是否提供 unit：

```
rpm -ql "$name" | grep -E '/systemd/(system|user)/'
```

只有得到真实 unit 名后才查询：

```
systemctl status <UNIT>
systemctl is-active <UNIT>
systemctl is-enabled <UNIT>
```

是否需要启动、启用、reload 或 restart 由题目终态和第 12 章方法决定。最后使用应用自身的版本、配置和业务接口完成验收。

## ⑨ 保留事务记录

```
dnf history
dnf history info <ID>
```

记录事务 ID、最终 NEVRA、配置处理和未完成的 live 验证。

## 验收矩阵

| 层次  | 合格条件             |
|-----|------------------|
| 包文件 | 包头、架构、脚本和签名状态已记录 |
| 事务  | 摘要符合目标，无绕过保护选项   |
| 数据库 | 最终 NEVRA 明确      |
| 文件  | 关键路径存在且归属正确      |
| 配置  | 旁置文件已处理或明确记录     |

| 层次   | 合格条件                     |
|------|--------------------------|
| unit | 提供关系、active、enabled 分别判断 |
| 功能   | 真实应用检查完成                 |
| 审计   | history 与变更记录可追溯         |

### 典型错误

- 把 `.rpm` 文件存在当成已安装；
- 只执行 `rpm -qpi`，未检查脚本和签名；
- 用 `rpm -ivh` 遇到依赖失败后添加 `--nodeps`；
- DNF 完成后不检查真实 NEVRA；
- 忽略 `.rpmnew/.rpmsave`；
- 看到 unit 文件就宣布服务可用。

### [本章收束] 包管理的终点不是 Complete!

本章建立的主线不是一串 RPM 选项，而是从 包身份与 NEVRA 出发，依次区分本地包文件和已安装数据库、建立文件归属、解释签名与 `rpm -V` 差异、审阅 DNF 事务，并最终推进到配置、unit 和真实功能。

### 工作方法：每次软件变更都保留六类证据

1. 变更前身份： 当前已安装 NEVRA、本地 RPM 包头、架构和信任状态；
2. 事务范围： DNF 摘要中将安装、升级和删除的完整集合；
3. 数据库终态： 事务后的真实 NEVRA 和 history 记录；
4. 文件终态： 关键路径、归属、属性差异和缺失状态；
5. 配置终态： `.rpmnew/.rpmsave` 的比较、合并与应用语法检查；
6. 功能终态： unit、进程、监听或应用真实接口的分层验收。

### 主要判断表

| 看到的证据                          | 能证明什么           | 不能证明什么           | 下一条高区分度证据                                           |
|--------------------------------|-----------------|------------------|-----------------------------------------------------|
| <code>rpm -q PACKAGE</code> 成功 | RPM 数据库存在匹配安装记录 | 文件未被修改、配置正确、服务可用 | <code>rpm -ql/-qc</code> 、关键路径和 <code>rpm -V</code> |
| <code>rpm -qf /path</code> 成功  | 某已安装包声明拥有该路径    | 谁最后修改、内容是否正确     | <code>stat</code> 、 <code>rpm -Vf</code> 、变更记录      |
| <code>rpm -V</code> 无输出        | 本次可检查属性未发现差异    | 业务配置符合要求、服务健康    | 应用语法、unit 和功能检查                                     |



| 看到的证据                               | 能证明什么             | 不能证明什么              | 下一条高区分度证据                                         |
|-------------------------------------|-------------------|---------------------|---------------------------------------------------|
| <code>rpm -V</code> 有输出             | 当前文件与数据库基线存在指定差异  | 差异是否恶意或需要恢复         | 文件类别、 <code>stat</code> 、 <code>diff</code> 、配置记录 |
| <code>rpmkeys --checksig</code> 可验证 | 包文件摘要和签名可按当前信任库验证 | 包适合当前系统、事务依赖可满足     | NEVRA、架构、Requires 和 DNF 摘要                        |
| DNF <code>Complete!</code>          | 包事务已经提交完成         | 配置迁移、unit 状态和业务功能正确 | 最终 NEVRA、旁置配置、应用与服务验收                             |
| 存在 <code>.rpmnew/.rpmsave</code>    | 升级保留了两份需要处理的配置证据  | 哪一份应直接替换另一份         | <code>diff</code> 、发行说明、应用语法检查                    |

### 向下一章交接

本章假定 DNF 已经能够访问合适的软件源，重点处理包文件、已安装数据库、文件差异和事务终态。第 17 章《DNF 仓库、模块流与包组》将继续回答：候选包从哪里来、仓库元数据如何加载、GPG key 如何配置、模块流和包组如何改变可选范围，以及 DNS、TLS、元数据和签名错误应从哪一层调查。